

APP Chat-RealTime

La Biblia del Proyecto: App de Mensajería TUP

A: Ulises, Nicolás, Pablo

De: Don Senior (Director de Proyecto)

Asunto: El Plan Maestro Completo.

Leer antes de programar.

1\ 🚀 Visión y Contexto del Proyecto

¿Qué estamos construyendo?

[cite_start]Vamos a construir un sistema de mensajería en tiempo real, onda WhatsApp[cite: 35]. El objetivo *no* es competir con WhatsApp, es demostrar que entendemos cómo armar una aplicación moderna, escalable y, sobre todo, **desacoplada**.

El "Por Qué" (Lo que el profe quiere ver)

[cite_start]El integrador pide .NET 9, microservicios, SignalR y JWT[cite: 35, 36]. Esto se traduce en:

1. **Microservicios:** ¿Saben separar responsabilidades? ¿Puede el servicio de Grupos caerse sin que se caiga el de Mensajes?
2. **Tiempo Real:** ¿Entienden la comunicación bidireccional? (SignalR) [cite_start][cite: 35, 56, 74].
3. [cite_start]**Seguridad:** ¿Saben cómo proteger un endpoint y validar un token (JWT)?[cite: 36, 89].

Nuestro plan está diseñado para responder "Sí" a todo esto, usando las herramientas más inteligentes para el trabajo.

2\ Stack Tecnológico (El "Qué Usamos" y "Por Qué")

Este es nuestro arsenal. Lo elegimos para ser eficientes y cumplir los requisitos.

- **Identidad (Auth): Supabase Auth**
 - **Contexto:** Es un golazo. Nos da un servicio de autenticación completo (registro, login, reseteo de pass) sin laburar. [cite_start]Genera los **Tokens JWT** que el TP pide [cite: 36, 89] y que usaremos para proteger *todas* nuestras APIs.
- **Base de Datos: Supabase (PostgreSQL Hosteado)**
 - **Contexto:** Acá está el truco. En vez de 3 bases de datos en Azure, usamos **UNA** sola base de datos de Supabase. Es más fácil de manejar y gratis. **PERO**, para simular microservicios, la vamos a dividir en **tres esquemas separados** (`usuarios` , `grupos` , `mensajería`). Cada servicio será dueño de su propio schema.
- **Backend (APIs): .NET 9**
 - [cite_start]**Contexto:** Es el requisito principal[cite: 35]. Crearemos 3 proyectos de API Web mínimos, prolijos y separados.
- **Hosting (APIs y UI): Azure (App Services y Static Web Apps)**
 - **Contexto:** Usamos el PaaS de Microsoft. Es fácil de desplegar (publicar desde Visual Studio), tiene un *tier* gratuito y nos permite tener URLs públicas para que la UI le pegue a las APIs.
- **Tiempo Real: SignalR**
 - [cite_start]**Contexto:** Es el requisito explícito del TP para "escribiendo...", "visto" y recibir mensajes[cite: 35, 56, 74, 84]. Esto vivirá dentro de la API de Mensajes de Pablo.

3\ Equipo, Roles y Propiedad (El "Quién es Quién")

Acá no hay grises. Cada uno es el **dueño absoluto** de su parte.

- **Ulises (El Arquitecto e Integrador)**

- [cite_start]**Propiedad:** `UI Mínima` (REQ-11)[cite: 94], `Usuarios.API`, y el schema `usuarios` en la DB.
 - **Responsabilidad:** Arma toda la FASE 1 (este setup). Desarrolla la UI, la conecta con Supabase Auth.js para el login. Crea el servicio de "sync" para guardar usuarios en nuestra DB. Integra el trabajo de Nicolás y Pablo en la UI.
 - **Nicolás (El Dueño de Grupos)**
 - [cite_start]**Propiedad:** `Grupos.API` (REQ-04, REQ-06) [cite: 60-62, 68-70] y el schema `grupos` en la DB.
 - **Responsabilidad:** Desarrollar *todos* los endpoints para crear grupos, eliminarlos, agregar miembros, sacar miembros y listar los grupos de un usuario. Su API no sabe nada de mensajes, solo de grupos.
 - **Pablo (El Dueño del Core Realtime)**
 - [cite_start]**Propiedad:** `Mensajes.API` (REQ-03, REQ-05, REQ-07, REQ-08, REQ-09) [cite: 55-58, 64-66, 72-74, 82-85, 87] y el schema `mensajería` en la DB.
 - **Responsabilidad:** Es el laburo más complejo. Armar el Hub de SignalR (protegido). [cite_start]Armar los endpoints para enviar mensajes (1:1 y grupal), listar historial (paginado), y *toda* la lógica de "está escribiendo..." [cite: 72] [cite_start]y "visto" [cite: 82][cite_start], incluyendo el listado de quién lo leyó[cite: 87].
-

4\. Arquitectura de la Solución (El Mapa)

Así se conecta todo. Entender esto es fundamental.

1. Flujo de Login:

- El **Usuario** abre la **UI** (en Azure Static Web App).
- La UI (con `supabase.js`) habla con **Supabase Auth** para hacer `signUp` o `signIn`.
- Supabase Auth devuelve un **Token JWT**.
- La UI *guarda* ese JWT.
- La UI llama a *nuestra* `POST /api/users/sync` (API de Ulises) enviando ese JWT.

- La `Usuarios.API` valida el JWT, extrae los datos del usuario (email, id) y lo guarda en la tabla `Perfiles` del schema `usuarios`.

2. Flujo de Chat:

- El Usuario (ya logueado) abre un chat.
- La UI (con el JWT) llama a `GET /api/mensajes/conversacion/{id}` (API de Pablo) para traer el historial.
- La UI (con el JWT) abre una conexión WebSocket con el **SignalR Hub** (en la API de Pablo).
- El Usuario escribe: la UI llama a `Hub.Invoke("NotificarEscribiendo")`.
- El Usuario envía: la UI llama a `POST /api/mensajes/directo` (API de Pablo).
- Esa API guarda en la DB (schema `mensajería`) y *avisa al Hub de SignalR*.
- El Hub *emite* el evento `MensajeRecibido` a los otros usuarios conectados.

5\ 🤝 Estrategia de Trabajo (Git y Comunicación)

Regla \#1: Simpleza. No necesitamos un Git Flow de 20 ramas.

- **Rama `main` : INTOCABLE.** Ulises, la configurarás como "protegida" en GitHub. Nadie puede pushear directo.
- **Rama `develop` : NUESTRA ÚNICA RAMA DE TRABAJO.** Acá pasa toda la acción.

Las 3 Reglas de Oro para Trabajar en `develop` :

1. **NUNCA `push` sin `pull`** : Antes de *empezar* a laburar y *antes* de subir tu código, siempre hacé `git pull --rebase` (o `git pull` si te complica el rebase). Esto te trae los cambios de los demás primero.
2. **COMUNICAR (Sobre-comunicar):** ¿Acabás de subir un cambio? Avisás al toque por el chat: "Muchachos, subí la base del `Grupos.API` a `develop`". ¿Estás por empezar algo grande? Avisás: "Voy a meterme con el Hub de SignalR".
3. **NO SUBIR CÓDIGO ROTO:** Si no compila en tu máquina, NO LO SUBAS. Estás rompiendo el proyecto para todos.

6\ 🏁 Roadmap - FASE 1: El Setup (La Misión de Ulises)

Tu misión, Ulises. El resto del equipo no empieza a programar hasta que no termines esto y des el "luz verde".

- **Paso 1. GitHub:**

- Crear el repositorio `tup-progi-v-integrador`.
- Invitar a Nicolás y Pablo como colaboradores.
- Crear la rama `develop` desde `main`.
- Ir a `Settings` \> `Branches` y proteger la rama `main`.

- **Paso 2. Supabase (El Corazón):**

- Crear una cuenta gratis en Supabase.
- Crear un nuevo Proyecto (ej. `tup-chat-app`).
- Ir a `Project Settings` \> `Database`: Buscar y copiar el **Connection String** (el que empieza con `postgres://...`).
- Ir a `Project Settings` \> `API`: Copiar la **URL** del proyecto, el `anon_key` (para tu UI) y, más abajo, el **JWT Secret** (es un string largo).
- Ir al `SQL Editor`: Correr estos comandos para crear nuestras "cajas" (schemas):

```
CREATE SCHEMA usuarios;  
CREATE SCHEMA grupos;  
CREATE SCHEMA mensajeria;
```

- **Paso 3. Azure (El Host):**

- Crear un Grupo de Recursos (ej. `RG-TUP-Mensajeria`).
- Crear los 4 servicios (Plan Gratuito F1/Free):
 1. **Static Web App:** `chat-app-tup` (para tu UI).
 2. **App Service:** `usuarios-api-tup` (para tu API).
 3. **App Service:** `grupos-api-tup` (para la API de Nico).
 4. **App Service:** `mensajes-api-tup` (para la API de Pablo).

- Juntar las 4 URLs públicas.
- **Paso 4. Estructura Local .NET:**
 - Clonar el repo.
 - `dotnet new sln -n ChatApp`
 - `dotnet new webapi -n Usuarios.API`
 - `dotnet new webapi -n Grupos.API`
 - `dotnet new webapi -n Mensajes.API`
 - `dotnet new classlib -n Shared.Kernel`
 - Ligar todo: `dotnet sln add ...` (los 4 proyectos).
- **Paso 5. Dependencias:**
 - Referenciar `Shared.Kernel` desde las 3 APIs: `dotnet add Usuarios.API reference Shared.Kernel`, etc.
 - Instalar el validador de JWT en `Shared.Kernel`: `dotnet add Shared.Kernel package Microsoft.AspNetCore.Authentication.JwtBearer`.
 - Instalar el driver de PostgreSQL en las 3 APIs: `dotnet add Usuarios.API package Npgsql.EntityFrameworkCore.PostgreSQL`, `dotnet add Grupos.API...`, etc.
- **Paso 6. Configuración (`appsettings.json`):**
 - En las 3 APIs, el `appsettings.json` debe tener esto (reemplazá con tus claves):

```
{
  "ConnectionStrings": {
    "DefaultConnection": "postgres://..." // El Connection String de S
upabase (Paso 2)
  },
  "Jwt": {
    "Issuer": "https://TU_PROYECTO_ID.supabase.co/auth/v1", // La
URL de tu proyecto + /auth/v1
    "Audience": "authenticated",
    "Key": "EL_JWT_SECRET_DE_SUPABASE" // El JWT Secret (Paso
2)
```

```
}  
}
```

- **Paso 7. Código de Seguridad (`Program.cs` de las 3 APIs):**
 - Agregar toda la lógica de seguridad (ver anexo v4).
 - **CORS:** Configurar para que *solo* acepte tu UI de Azure y `localhost`.
 - **JWT:** Configurar para que valide los tokens de Supabase (usando los `appsettings`).
 - **Rate Limiting:** Agregar el límite de requests (REQ-10).
- **Paso 8. Primer Push:**
 - `git add` , `git commit -m "feat: setup inicial v4 (Supabase + 3 APIs)"` , `git push -u origin develop` .
- **Paso 9. Configurar Deploy (El "Go"):**
 - Configurar "Publicar" en las 3 APIs para que apunten a sus App Services en Azure.
 - Hacer "publish" de las 3 APIs (vacías) para verificar que las URLs de Azure funcionan y no dan 404.

¡FIN DE FASE 1! Avisás: "Muchachos, el esqueleto está en `develop` . Usamos Supabase para DB y Auth. Las APIs están vivas en Azure. Hagan `git pull` y empiecen. Cada uno en su schema".

7\ 🖥️ Roadmap - FASE 2: El Desarrollo en Paralelo

Todos hacen `git pull origin develop` . Cada uno abre la solución `ChatApp.sln` pero solo labura en **su** proyecto (`.csproj`) y **su** schema.

RUTA DE ULISES (Tu Laburo)

1. Frontend (UI):

- Elegir (React/Vue/Blazor) y armar el proyecto.
- Instalar `supabase-js` : `npm install @supabase/supabase-js` .
- Implementar `supabase.auth.signUp()` y `supabase.auth.signInWithPassword()` para el login/registro.
- Implementar `supabase.auth.getSession()` para sacar el JWT (access token).

- Armar el "interceptor" de Axios para poner ese token en el header `Authorization: Bearer ...` en CADA llamada a nuestras APIs.
- Armar el `staticwebapp.config.json` para la seguridad CSP (ver anexo).

2. Backend (`Usuarios.API`):

- Crear tu `UsuariosDbContext` .
- **CLAVE:** En el `OnModelCreating` de tu DbContext, poner: `modelBuilder.HasDefaultSchema("usuarios");` .
- Modelar tu tabla `Perfiles` (ej. `Id` (UUID), `AuthId` (el 'sub' de Supabase), `Email` , `Nombre`).
- Implementar `POST /api/users/sync` (Protegido con `[Authorize]`).
- Implementar `GET /api/users/me` (Protegido con `[Authorize]`).
- **Push & Deploy:** `pull` , `push` , `publish` .

3. Integración (UI):

- Llamar a tu `/api/users/sync` después del login.
- Empezar a consumir las APIs de Nico y Pablo (cuando estén listas) para renderizar la lista de chats, los mensajes, etc.
- Conectar el cliente de SignalR al Hub de Pablo.

RUTA DE NICOLÁS (`Grupos.API`)

1. Backend (`Grupos.API`):

- Verificar que `Npgsql.EntityFrameworkCore.PostgreSQL` esté instalado.
- Crear tu `GruposDbContext` .
- **CLAVE:** En el `OnModelCreating` , poner: `modelBuilder.HasDefaultSchema("grupos");` .
- [cite_start]Modelar tus tablas `Grupos` y `MiembrosGrupo` (REQ-04) [cite: 60-62].
- Implementar *toda* tu API (Crear grupo, agregar miembro, listar grupos) usando `[Authorize]` en CADA endpoint.
- **Push & Deploy:** `pull` , `push` , y "Publicar" a tu App Service (`grupos-api-tup...`).
- Avisar a Ulises: "Los endpoints de Grupos están en la nube".

RUTA DE PABLO (`Mensajes.API` y SignalR)

1. Backend (**Mensajes.API**):

- Verificar `Npgsql` e instalar `Microsoft.AspNetCore.SignalR` .
- Crear tu `MensajesDbContext` .
- **CLAVE:** En el `OnModelCreating` , poner: `modelBuilder.HasDefaultSchema("mensajería");` .
- [cite_start]Modelar tus tablas `Mensajes` y `LecturasMensaje` (REQ-03, REQ-08) [cite: 55-58, 82-85].
- Crear el `ChatHub.cs` (protegido con `[Authorize]`) y mapearlo en `Program.cs` .
- [cite_start]Implementar *toda* tu API y lógica de Hub (Enviar, listar, "visto", "escribiendo") [cite: 55-58, 64-66, 72-74, 82-85, 87].
- **Push & Deploy:** `pull` , `push` , y "Publicar" a tu App Service (`mensajes-api-tup...`).
- Avisar a Ulises: "El Hub y la API de Mensajes están listos".

8\ El Contrato (Definición de APIs v4.0)

Esta es la interfaz. Es nuestro contrato. No se cambia sin avisar a los 3.

Usuarios.API (Ulises)

- `POST /api/users/sync` (Protegido)
 - **Acción:** Sincroniza el usuario del token de Supabase con la DB local.
 - **Respuesta (200):** `{ "id" (nuestro UUID), "authId", "email", "nombre" }`
- `GET /api/users/me` (Protegido)
 - **Respuesta (200):** `{ "id", "authId", "email", "nombre" }`

Grupos.API (Nicolás)

- `POST /api/grupos` (Protegido) (Body: `{ "nombre": "...", "miembrosIds": ["id_local_1"] }`)
- `GET /api/grupos/mis-grupos` (Protegido)
- `GET /api/grupos/{id}/miembros` (Protegido)
- `POST /api/grupos/{id}/miembros` (Protegido) (Body: `{ "userId": "id_local_del_usuario" }`)
- `DELETE /api/grupos/{id}/miembros/{userId}` (Protegido)

Mensajes.API (Pablo)

- `POST /api/mensajes/directo` (Protegido) (Body: `{ "destinatarioid": "id_local", "contenido": "..." }`)
- `POST /api/mensajes/grupo` (Protegido) (Body: `{ "grupoid": "...", "contenido": "..." }`)
- `GET /api/mensajes/conversacion/{userId}?page=1&pageSize=20` (Protegido, Paginado)
- `GET /api/mensajes/grupo/{grupoid}?page=1&pageSize=20` (Protegido, Paginado)
- `POST /api/mensajes/{mensajeld}/visto` (Protegido)
- [cite_start] `GET /api/mensajes/{mensajeld}/lecturas` (Protegido) (REQ-09) [cite: 87]

ChatHub (Pablo)

- **Cliente Invoca (Ulises llama a esto):**
 - `NotificarEscribiendo(string conversacionId)`
- **Servidor Emite (Ulises escucha esto):**
 - `MensajeRecibido(MensajeDto mensaje)`
 - `UsuarioEscribiendo(string conversacionId, string nombreUsuario)`
 - `MensajeLeido(string mensajeld, string userId)`

9\. FASE 3: Integración, Pruebas y Entrega

1. **Integración (Días Previos):** Ulises junta todo en la UI. Empezamos a debuggear en conjunto ("El CORS me bloquea", "Tu hub no me deja conectar", "El ID de usuario no coincide").
2. [cite_start] **Documentación (En Paralelo):** Empezamos a armar el **Informe PDF (Formato APA)** y la **PPT** para el coloquio[cite: 21, 22]. No dejen esto para el final.
3. **Entrega Final:** Ulises abre el único Pull Request del proyecto: de `develop` -> `main` . Los 3 aprueban. Se mergea. [cite_start]Se presenta la app funcionando[cite: 23].

10\. Anexos de Código Crítico

`appsettings.json` (Para las 3 APIs)

```
{
  "ConnectionStrings": {
```

```

    "DefaultConnection": "postgres://..." // Tu Connection String de Supabas
e
  },
  "Jwt": {
    "Issuer": "https://TU_PROYECTO_ID.supabase.co/auth/v1", // Reemplazar
    "Audience": "authenticated",
    "Key": "EL_JWT_SECRET_DE_SUPABASE" // Reemplazar
  }
}

```

Program.cs (Validación de JWT de Supabase)

```

var builder = WebApplication.CreateBuilder(args);
var config = builder.Configuration;

// 1. CORS
var uiAppUrl = "<https://chat-app-tup.azurewebsites.net>"; // UI de Ulises
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowSpecificOrigins",
        policy =>
        {
            policy.WithOrigins(uiAppUrl, "<http://localhost:3000>", "<http://loca
lhost:5173>") // UI Azure y Locales
                .AllowAnyHeader()
                .AllowAnyMethod()
                .AllowCredentials(); // Importante para SignalR
        });
});

// 2. RATE LIMITING
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("fixed", limiter =>
    {
        limiter.PermitLimit = 100;
        limiter.Window = TimeSpan.FromMinutes(1);
    });
});

```

```

});

// 3. AUTENTICACIÓN (JWT de Supabase)
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = config["Jwt:Issuer"],
            ValidAudience = config["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(config["Jwt:Key"]))
        };

        // ¡PARA PABLO! (Autenticar SignalR)
        options.Events = new JwtBearerEvents
        {
            OnMessageReceived = context =>
            {
                var accessToken = context.Request.Query["access_token"];
                var path = context.HttpContext.Request.Path;
                if (!string.IsNullOrEmpty(accessToken) && (path.StartsWithSegments("/chathub")))
                {
                    context.Token = accessToken;
                }
                return Task.CompletedTask;
            }
        };
    });

var app = builder.Build();

```

```
app.UseCors("AllowSpecificOrigins");
app.UseRateLimiter();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers().RequireRateLimiting("fixed");
// Pablo: app.MapHub<ChatHub>("/chathub").RequireAuthorization();
app.Run();
```

DbContext (Separación por Schemas)

Para **UsuariosDbContext** (Ulises):

```
public class UsuariosDbContext : DbContext
{
    public UsuariosDbContext(DbContextOptions<UsuariosDbContext> options) : base(options) { }
    // public DbSet<Perfil> Perfiles { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema("usuarios");
    }
}
```

(Nicolás y Pablo: hacer lo mismo con "grupos" y "mensajería" en sus DbContexts)

staticwebapp.config.json (CSP para Supabase, UI de Ulises)

```
{
  "headers": {
    "Content-Security-Policy": "default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; connect-src 'self' https://TU_URL_SUPABASE.supabase.co wss://TU_URL_SUPABASE.supabase.co <https://usuarios-api-tup.azurewebsites.net> <https://grupos-api-tup.azurewebsites.net> <https://mensajes-api-tup.azurewebsites.net> wss://mensajes-api-tup.azurewebsites.net;"
  },
}
```

```
"navigationFallback": { "rewrite": "/index.html" }  
}
```

Luz verde. A laburar.